

Computer Vision Mini-Project

February 2026

1 General Guidelines

This describes the Computer Vision assessed practical. The main goal is to explore deep learning approach to image processing, segmentation, and unsupervised knowledge extraction.

There are two tasks to this assignment, and you must do all two for full marks. You must do this project with a partner and produce a single report and the developed code.

The assignment is estimated to take 40 hours coding/test and 10 hours report writing per person.

Split the work so that each partner can do most work independently (i.e. share the work rather than duplicate it).

1.1 Report Guidelines

The report should be up to 10 pages long excluding references and with a minimum font size of 11pt and named as <student-number-1><student-number-2>.pdf. It should clearly describe the aims, the work done, and discuss:

- The augmentations or preprocessing steps eventually implemented, describe the algorithm and implementation details and dataset creation steps (Task 2).
- The network design and implementation.
- How you trained each of the network on the dataset.
- Provide relevant metrics (IoU, Dice, pixel accuracy, precision, recall, etc.) on the testing sets, summarize results in tables or charts.
- Show example segmentation/classification outputs (especially interesting failures or successes).
- Analyze key challenges: e.g., class imbalance, small objects, etc.
- Compare the performance of different approaches, try to analyze the reason.
- Please provide a detailed description of each member's responsibilities. If a particular task is carried out by both members, please specify which portions each person completed.

1.2 Code Guidelines

Please create a single zip file containing the code that your team wrote named <student-number-1><student-number-2>.zip. Do not include the code that was downloaded from other web sites, but include a statement about what code was used and where it came from. Please clearly comment the code written by the team and provide a *requirements.txt* file.

1.3 Submission Guidelines

The deadline for submission is 12:00 am, 09/04/2026. Please make your submission anonymous (i.e., no identifying information in the PDF). Name the submitted folder, containing both code and report as: <student-number-1><student-number-2>. Submit your final folder online using Learn. Details will be circulated later. The proportion of marks are explained in the Learn page.

1.4 Publication of Solutions

We will not publish a solution set of code. You may make public your solution but only 4 weeks after the submission date. Making the solutions public before then will create suspicions about why you made them public.

Good Scholarly Practice:

- Please remember the University requirement as regards all assessed work. Details about this can be found at: <https://informatics.ed.ac.uk/taught-students/all-students/your-studies/academic-misconduct>
- Furthermore, you are required to take reasonable measures to protect your assessed work from unauthorised access. For example, if you put any such work on a public repository then you must set access permissions appropriately (generally permitting access only to yourself, or your group in the case of group practicals).

2 Assignment

Image segmentation is the process of partitioning an image into multiple segments (sets of pixels). The goal of segmentation is to simplify or change the representation of an image into something more meaningful and easier to analyze. In many computer vision systems, segmentation constitutes a crucial step towards object recognition, scene understanding, medical analysis, and more. Image classification is the process of assigning an image its class based on different visual characteristics.

You will be given a dataset, consisting of a set of images. The dataset is already split into training, validation, and testing folders for you. You will need to implement segmentation and classification models.

2.1 Dataset

You will be provided with a split version of the [Puma Dataset](#) (see for additional details), the data to be used in the project can be found [here](#), the test set is provided and **must not be used** for model training.

2.2 Task 1: Tissue Segmentation

The dataset contains labeled images and corresponding ground-truths that indicate which pixels belong to which object or category. Make sure you familiarize yourself with the dataset structure (train/validation/test splits, image dimensions, mask formats, label classes, etc.). The images are in .tif format while the labels are provided in .GeoJSON format (you can visualize them with the open-source pathology image viewer [Qupath](#)).

2.2.1 Segmentation network design and implementation

Given the dataset with N classes of objects, you can simply consider image segmentation as a pixel-wise classification task. The input to your segmentation model should be an image with the size of $(H, W, 3)$, and the output will be a multi-class image mask with the size of (H, W) . For this task please pre-process the dataset to only consider three classes *Tissue Tumor*, *Tissue Stroma*, and *Other*. *Other* consists of the combination of *Tissue Blood Vessel*, *Tissue Epidermis*, *Tissue White Background* and *Tissue Necrosis*.

- a) **UNet-based end-to-end segmentation neural network:** UNet is a popular architecture for image segmentation. Its encoder-decoder structure, with skip connections between the encoder and decoder, helps preserve spatial information lost during down-sampling and improves segmentation accuracy. Design your own UNet, and train it on the dataset. The input to the UNet model should be the image, and the output will be the image segmentation mask.
- b) **Autoencoder pre-training for segmentation:** An autoencoder learns to compress and reconstruct images. For this task, you should first train an autoencoder on the raw images (without labels) to reconstruct the image itself. You can then make the autoencoder's encoder fixed, use its features to train another decoder for the image segmentation task. The hypothesis is that pre-training helps the network learn relevant low-level and mid-level features, accelerating subsequent segmentation training and potentially improving accuracy.

2.2.2 Evaluation and comparison

- a) Quantitatively and qualitatively evaluate the segmentation performance on the provided testing set.
 - Use metrics such as Intersection over Union (IoU), Dice coefficient (F1 score for segmentation), or Pixel Accuracy, as appropriate for the dataset and its multi-class nature.
 - The baseline's average Dice coefficient across the three categories is 0.4670, and your best-performing method is supposed to be above the baseline (N.B. we are more interested in the motivations behind your implementation choices rather than a big improvement over the baseline mean Dice coefficient).
 - The baseline has $\sim 125M$ trainable parameters. Report the trainable parameter of your own solutions and comment on such a comparison.
- b) Compare the performance of your two segmentation models. Which architecture or approach achieves the best metrics? How does pre-training influence the results?
- c) Given the dataset class imbalance which consideration could be made on implementation choices and obtained performance?

2.3 Task 2: Nuclei Classification

Using the same dataset splits as in Task 1 this second part will focus on nuclei. Again the images are .tif and the labels are provided in .GeoJSON (you can visualize them with the open-source pathology image viewer [Qupath](#)).

We have provided in the folder *Task2_Test_Set* the test set for the Task 2 part. It contains image patches containing nuclei from 3 classes (i.e., *Tumor*, *Lymphocyte*, and *Histiocyte*)

extracted from the same test test of Task 1. The patches have been extracted by cropping a 100×100 square centered in the nuclei center and zero-padded if needed.

2.3.1 Dataset Creation

For this task you will need to create your own training and validation sets and discuss any specific choice you have made in creating such datasets.

- **Training set:** extract 2500 patches per class from the originally provided training set folder.
- **Validation set:** extract 700 patches per class from the originally provided validation set folder.
- **Contrastive set:** decide how many patches per class from the originally provided training set folder you will like and extract them (be sure to prevent data leakage).

2.3.2 Classification network design and implementation

Given the dataset with 3 classes of objects, the input to your classification model should be an image with the size of (H, W, 3), and the output will be an integer representing the class. For this task please only consider *Tumor*, *Lymphocyte*, and *Histiocyte* as valid classes.

- a) **End-to-end classification neural network:** Select and implement your classification network, and train it on the training dataset you have created in task 2.3.1. The input to the classification model should be the image, and the output will be the corresponding nucleus class.
- b) **Contrastive learning pre-training for classification:** Contrastive learning trains models to group similar data points closer in an embedding space and push dissimilar ones apart. For this task, you should first implement a contrastive learning pre-training strategy (you can choose the one you prefer) using the Contrastive set you created in task 2.3.1. You can then make the encoder fixed, use its features to train another classification head for the nuclei classification task. The hypothesis is that contrastive learning helps the network learn relevant class differences on large unlabeled data and subsequently potentially improving accuracy for small annotated datasets.

2.3.3 Evaluation and comparison

- a) Quantitatively and qualitatively evaluate the classification performance on the provided testing set.
 - Use metrics such as precision and recall as appropriate for the dataset and its multi-class nature.
 - The baseline’s average accuracy across the 3 categories is 0.7083, and your best-performing method is supposed to be above the baseline (N.B. we are more interested in the motivations behind your implementation choices rather than a big improvement over the baseline mean accuracy).
 - The baseline has $\sim 5M$ trainable parameters. Report the trainable parameter of your own solutions and comment on such a comparison.
 - Evaluate the quality of the obtained latent space for the contrastive learning-based part. You could use both visual or numerical evaluations.

- b) Compare the performance of your two classification models. Which architecture or approach achieves the best metrics? How does pre-training influence the results?
- c) Does the nature of the samples (i.e., primary vs metastatic) impact performance? Please comment.